

Lab 2 - Data wrangling

John Q. Student

1497-06-24

```
library(tidyverse)
midwest
```

```
# A tibble: 437 x 28
  PID county state area poptotal popdensity popwhite popblack popamerindian
  <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int>
1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98
2 562 ALEXAN~ IL 0.014 10626 759 7054 3496 19
3 563 BOND IL 0.022 14991 681. 14477 429 35
4 564 BOONE IL 0.017 30806 1812. 29344 127 46
5 565 BROWN IL 0.018 5836 324. 5264 547 14
6 566 BUREAU IL 0.05 35688 714. 35157 50 65
7 567 CALHOUN IL 0.017 5322 313. 5298 1 8
8 568 CARROLL IL 0.027 16805 622. 16519 111 30
9 569 CASS IL 0.024 13437 560. 13384 16 8
10 570 CHAMPA~ IL 0.058 173025 2983. 146506 16559 331
# i 427 more rows
# i 19 more variables: popasian <int>, popother <int>, percwhite <dbl>,
# percblack <dbl>, percamerindian <dbl>, percasian <dbl>, percother <dbl>,
# popadults <int>, perchs <dbl>, percollege <dbl>, percprof <dbl>,
# poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
# percchildbelowpovert <dbl>, percadultpoverty <dbl>,
# percelderlypoverty <dbl>, inmetro <int>, category <chr>
```

Questions

Part 1

Question 1

```
midwest %>%  
  count(state, sort = TRUE)
```

```
# A tibble: 5 x 2  
  state      n  
  <chr> <int>  
1 IL      102  
2 IN       92  
3 OH       88  
4 MI       83  
5 WI       72
```

The state with the highest number of counties is **Illinois**, and the state with the lowest number of counties is **WI**.

Question 2

```
midwest %>%  
  count(county) %>%  
  filter(n == n_distinct(midwest$state))
```

```
# A tibble: 3 x 2  
  county      n  
  <chr>    <int>  
1 CRAWFORD     5  
2 JACKSON      5  
3 MONROE       5
```

These are the county names that appear in every state in the dataset.

Question 3

a

```
midwest %>%
  filter(poptotal / area > 25000) %>%
  mutate(pop_density = poptotal / area) %>%
  select(county, state, pop_density, poptotal, area) %>%
  arrange(desc(pop_density))
```

A tibble: 437 x 5

	county	state	pop_density	poptotal	area
	<chr>	<chr>	<dbl>	<int>	<dbl>
1	COOK	IL	88018397.	5105067	0.058
2	MILWAUKEE	WI	63951667.	959275	0.015
3	WAYNE	MI	60333914.	2111687	0.035
4	CUYAHOGA	OH	54313077.	1412140	0.026
5	DU PAGE	IL	39083300	781666	0.02
6	MARION	IN	34659087.	797159	0.023
7	HAMILTON	OH	34649120	866228	0.025
8	FRANKLIN	OH	28277559.	961437	0.034
9	MACOMB	MI	25621429.	717400	0.028
10	LUCAS	OH	22017190.	462361	0.021

i 427 more rows

b

```
midwest %>%
  mutate(pop_density = poptotal / area) %>%
  filter(pop_density == max(pop_density)) %>%
  select(county, state, pop_density, poptotal, area)
```

A tibble: 1 x 5

	county	state	pop_density	poptotal	area
	<chr>	<chr>	<dbl>	<int>	<dbl>
1	COOK	IL	88018397.	5105067	0.058

Question 4

```
midwest %>%
  mutate(pop_density = poptotal / area) %>%
  summarise(
    median_density = median(pop_density),
    Q1 = quantile(pop_density, 0.25),
    Q3 = quantile(pop_density, 0.75)
  )
```

```
# A tibble: 1 x 3
  median_density    Q1      Q3
      <dbl>    <dbl>   <dbl>
1    1156208. 622407. 2330000
```

The distribution of population density is right-skewed. A typical county has a population density around the 1156208. The middle 50% of counties fall between 622407.4 and 2330000.

Question 5

```
midwest %>%
  mutate(metro = if_else(inmetro == 1, "Yes", "No")) %>%
  count(state, metro) %>%
  group_by(state) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 10 x 4
# Groups:   state [5]
  state metro     n prop
  <chr> <chr> <int> <dbl>
1 IL    No      74 0.725
2 IL    Yes     28 0.275
3 IN    No     55 0.598
4 IN    Yes     37 0.402
5 MI    No     58 0.699
6 MI    Yes     25 0.301
7 OH    No     48 0.545
8 OH    Yes     40 0.455
9 WI    No     52 0.722
10 WI   Yes     20 0.278
```

Question 6

a.

```
midwest %>%  
  filter(  
    percbelowpoverty > 30,  
    percollege < 15  
  ) %>%  
  select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 3 x 4  
  county      state percbelowpoverty percollege  
  <chr>      <chr>          <dbl>      <dbl>  
1 ALEXANDER  IL              32.2       11.2  
2 PULASKI    IL              30.2       14.4  
3 MENOMINEE WI              48.7        7.34
```

b.

```
midwest %>%  
  filter(  
    percbelowpoverty > 15,  
    percollege > 35  
  ) %>%  
  select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 4 x 4  
  county      state percbelowpoverty percollege  
  <chr>      <chr>          <dbl>      <dbl>  
1 CHAMPAIGN IL              15.6       41.3  
2 JACKSON   IL              28.4       36.6  
3 MONROE    IN              19.4       37.7  
4 INGHAM    MI              16.6       36.8
```

c.

```
midwest %>%
  filter(
    (percbelowpoverty > 30 & percollege < 15) |
    (percbelowpoverty > 15 & percollege > 35)
  ) %>%
  select(county, state, percbelowpoverty, percollege)
```

```
# A tibble: 7 x 4
  county      state percbelowpoverty percollege
  <chr>      <chr>          <dbl>      <dbl>
1 ALEXANDER  IL              32.2       11.2
2 CHAMPAIGN IL              15.6       41.3
3 JACKSON    IL              28.4       36.6
4 PULASKI    IL              30.2       14.4
5 MONROE     IN              19.4       37.7
6 INGHAM     MI              16.6       36.8
7 MENOMINEE WI              48.7        7.34
```

d.

```
midwest_updated <- midwest %>%
  mutate(
    potential_outlier = if_else(
      (percbelowpoverty > 30 & percollege < 15) |
      (percbelowpoverty > 15 & percollege > 35),
      "Yes",
      "No"
    )
  ) %>%
  select(
    county,
    state,
    percbelowpoverty,
    percollege,
    potential_outlier
  ) %>%
  arrange(potential_outlier)

midwest_updated
```

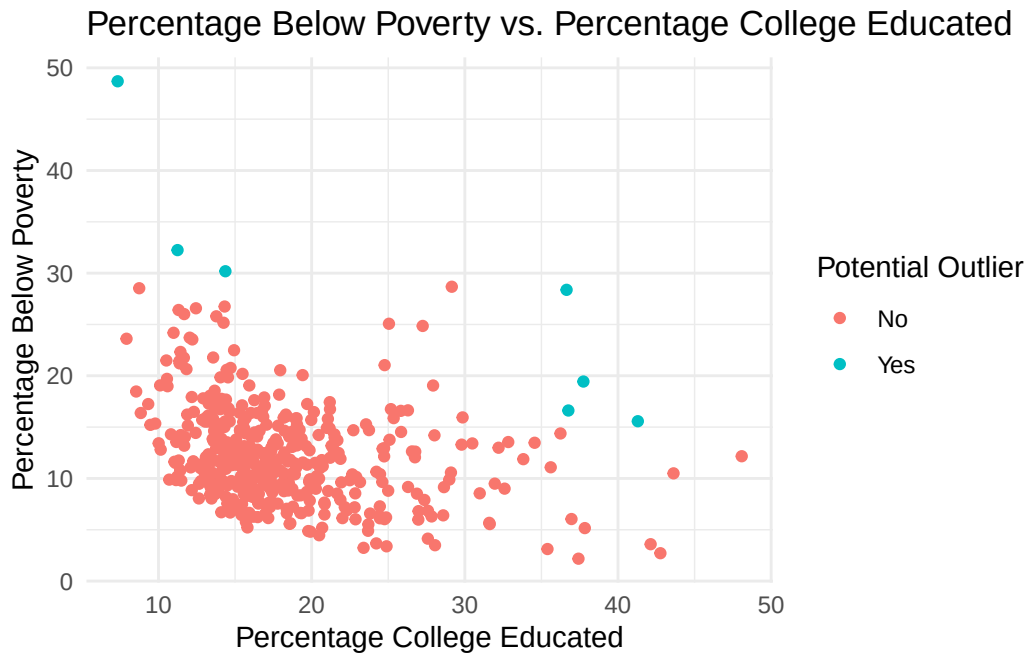
```
# A tibble: 437 x 5
```

	county	state	percbelowpoverty	percollege	potential_outlier
	<chr>	<chr>	<dbl>	<dbl>	<chr>
1	ADAMS	IL	13.2	19.6	No
2	BOND	IL	12.1	17.0	No
3	BOONE	IL	7.21	17.3	No
4	BROWN	IL	13.5	14.5	No
5	BUREAU	IL	10.4	18.9	No
6	CALHOUN	IL	15.1	11.9	No
7	CARROLL	IL	11.7	16.2	No
8	CASS	IL	13.9	14.1	No
9	CHRISTIAN	IL	11.7	13.6	No
10	CLARK	IL	12.0	15.1	No

i 427 more rows

e.

```
ggplot(
  midwest_updated,
  aes(
    x = percollege,
    y = percbelowpoverty,
    color = potential_outlier
  )
) +
  geom_point() +
  labs(
    title = "Percentage Below Poverty vs. Percentage College Educated",
    x = "Percentage College Educated",
    y = "Percentage Below Poverty",
    color = "Potential Outlier"
  ) +
  theme_minimal()
```



Question 7

a.

```
state_population <- midwest %>%
  group_by(state) %>%
  summarise(total_population = sum(poptotal)) %>%
  arrange(desc(total_population))

state_population
```

```
# A tibble: 5 x 2
  state total_population
  <chr>         <int>
1 IL           11430602
2 OH           10847115
3 MI            9295297
4 IN            5544159
5 WI            4891769
```

b.

```
state_population %>%
  mutate(population_proportion = total_population / sum(total_population)) %>%
  arrange(desc(population_proportion))
```

```
# A tibble: 5 x 3
  state total_population population_proportion
  <chr>         <int>             <dbl>
1 IL           11430602             0.272
2 OH           10847115             0.258
3 MI            9295297             0.221
4 IN            5544159             0.132
5 WI            4891769             0.116
```

c.

The most populous Midwestern state is the one at the top of the table in part (b), and its proportion gives the percent of the Midwest population living there. The least populous state is the one at the bottom of the table, and its proportion gives the smallest share of the Midwest population.

Question 8

```
state_poverty <- midwest %>%  
  group_by(state) %>%  
  summarise(mean_percbelowpoverty = mean(percbelowpoverty))  
  
state_poverty
```

```
# A tibble: 5 x 2  
  state mean_percbelowpoverty  
  <chr>          <dbl>  
1 IL             13.1  
2 IN             10.3  
3 MI             14.2  
4 OH             13.0  
5 WI             11.9
```

```
state_poverty %>%  
  arrange(mean_percbelowpoverty)
```

```
# A tibble: 5 x 2  
  state mean_percbelowpoverty  
  <chr>          <dbl>  
1 IN             10.3  
2 WI             11.9  
3 OH             13.0  
4 IL             13.1  
5 MI             14.2
```

The state with the lowest average percentage below poverty is the one at the top of the table, and the state with the highest average percentage below poverty is the one at the bottom.

Part 2

Question 9

```
df <- tibble(  
  var_1 = c(10, 20, 30, 40, 50),  
  var_2 = c("Pizza", "Burger", "Pizza", "Pizza", "Burger"),  
  var_3 = c("Apple", "Apple", "Pear", "Pear", "Banana")  
)  
  
df
```

```
# A tibble: 5 x 3  
  var_1 var_2 var_3  
  <dbl> <chr> <chr>  
1     10 Pizza Apple  
2     20 Burger Apple  
3     30 Pizza Pear  
4     40 Pizza Pear  
5     50 Burger Banana
```

a.

```
df |>  
  arrange(var_2)
```

```
# A tibble: 5 x 3  
  var_1 var_2 var_3  
  <dbl> <chr> <chr>  
1     20 Burger Apple  
2     50 Burger Banana  
3     10 Pizza Apple  
4     30 Pizza Pear  
5     40 Pizza Pear
```

This code sorts the rows of `df` by `var_2` in ascending alphabetical order. The `arrange()` function changes the order of the rows, but it does not group the data or calculate any summaries.

b.

```
df |>
  group_by(var_2)

# A tibble: 5 x 3
# Groups:   var_2 [2]
  var_1 var_2 var_3
  <dbl> <chr> <chr>
1     10 Pizza Apple
2     20 Burger Apple
3     30 Pizza Pear
4     40 Pizza Pear
5     50 Burger Banana
```

This code groups the data by `var_2`, meaning observations with the same value of `var_2` are placed in the same group. Unlike `arrange()`, `group_by()` does not sort the rows for display purposes; instead, it prepares the data for grouped operations like `summarize()` or `mutate()`.

c.

```
df |>
  group_by(var_2) |>
  summarize(mean_var_1 = mean(var_1))

# A tibble: 2 x 2
  var_2 mean_var_1
  <chr>      <dbl>
1 Burger      35
2 Pizza     26.7
```

This pipeline groups the data by `var_2` and then calculates the mean of `var_1` for each group. The result is a new data frame with one row for each value of `var_2` and a column showing the average of `var_1` within that group.

d.

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by var_2 and var_3.
i Output is grouped by var_2.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(var_2, var_3))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 4 x 3
# Groups:   var_2 [2]
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple        20
2 Burger Banana       50
3 Pizza  Apple        10
4 Pizza  Pear         35
```

This pipeline groups the data by both `var_2` and `var_3`, then calculates the mean of `var_1` for each combination of those two variables. The message indicates that the output is still grouped by `var_2`, because `summarize()` drops only the last level of grouping by default.

e.

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1), .groups = "drop")
```

```
# A tibble: 4 x 3
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple        20
2 Burger Banana       50
3 Pizza  Apple        10
4 Pizza  Pear         35
```

This pipeline also calculates the mean of `var_1` for each combination of `var_2` and `var_3`, but the `.groups = "drop"` argument removes all grouping from the output. Unlike part (d), the result is no longer grouped after the summary is created.

f.

```
df |>
  group_by(var_2, var_3) |>
  summarize(mean_var_1 = mean(var_1), .groups = "drop")
```

```
# A tibble: 4 x 3
  var_2 var_3 mean_var_1
  <chr> <chr>      <dbl>
1 Burger Apple        20
2 Burger Banana       50
3 Pizza  Apple        10
4 Pizza  Pear         35
```

This pipeline creates a smaller summarized data frame with one row for each combination of `var_2` and `var_3`, along with the mean of `var_1` for that group.

```
df |>
  group_by(var_2, var_3) |>
  mutate(mean_var_1 = mean(var_1))
```

```
# A tibble: 5 x 4
# Groups:   var_2, var_3 [4]
  var_1 var_2 var_3 mean_var_1
  <dbl> <chr> <chr>      <dbl>
1    10 Pizza Apple        10
2    20 Burger Apple       20
3    30 Pizza Pear        35
4    40 Pizza Pear        35
5    50 Burger Banana      50
```

This pipeline keeps all original rows but adds a new column called `mean_var_1`, which gives the mean of `var_1` within each `var_2` and `var_3` group.

The outputs are different because `summarize()` reduces the data to one row per group, while `mutate()` keeps every original row and adds the group-level calculation alongside it.